

CS212 Project

Hybrid Encryption Based Secure Client-Server System

Features:

- RSA Key Exchange
- Symmetric Session Encryption
- Authentication
- Multi-Client Threaded Server (Socket Programming)
- Role-Based Access Control (Admin / User)

Coding Language:

- Python

NAME : MACHHI PRANAY BHARATBHAI
ROLL NO. : CSE 2403122

ABOUT THE PROJECT

- Normal Server-Client communication using Socket Programming has Many Problems like :
 - Password interception
 - Data leakage
 - Unauthorized access
 - Man-in-the-middle attacks
 - Plaintext communication
- Secure Client-Server communication using:
 - Hybrid Encryption :
 - (Asymmetric Encryption [RSA] + Symmetric Encryption)

Why Hybrid Encryption :

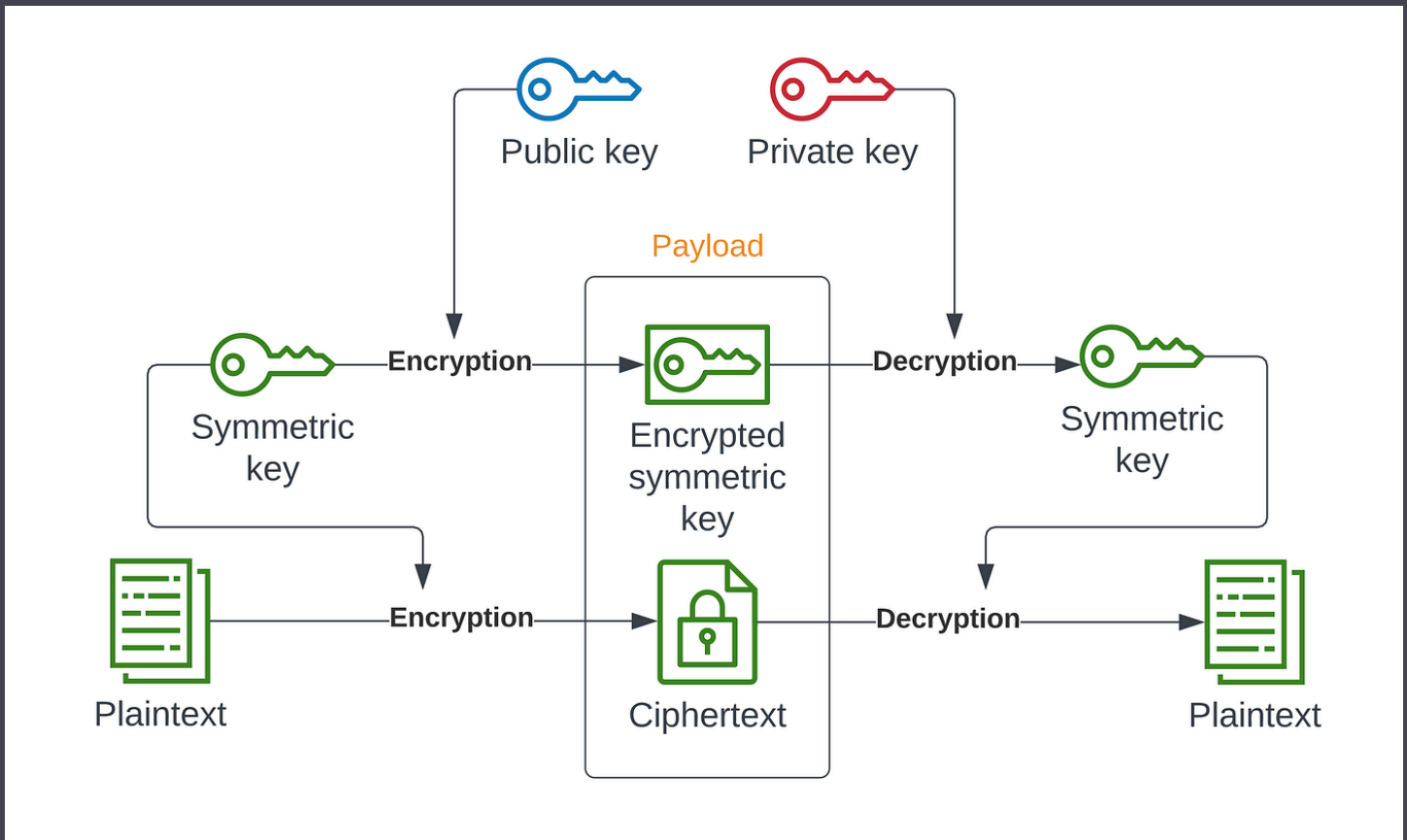
Asymmetric Encryption : to Share Common Symmetric key securely (Session Key)

Symmetric Encryption : to make sharing the data faster as Asymmetric Encryption is slow for larger messages

Hybrid Encryption : Asymmetric Encryption + Symmetric Encryption (better approach)

HYBRID ENCRYPTION

- Asymmetric Encryption + Symmetric Encryption (better approach)



Combining both gives:

- Security (RSA)
- Speed (Symmetric Encryption)

System Architecture

1. RSA Public Key Shared (Server to Client)
2. Session Key Generated (Inside Client System)
3. Session Key Encrypted using RSA and sent (Client to Server)
4. Authentication Phase (Database) [no specific database is used for this project]

Two dummy accounts are hardcoded :

- Username : user
 - Password : user123
-
- Username : admin
 - Password : admin123

5. Encrypted Communication Starts using session key

Session Key Exchange

Step 1 : Server generates RSA keys

public_key , private_key

Step 2 : Client creates Session key (random string)

Step 3 : Client encrypts session key using RSA public key.

Step 4 : Server decrypts using private key.

```
Pranay Machhi@Lenovo MINGW64 ~/OneDrive/Desktop/Coding/Programs 2.0/Socket_P
rogramming_PYTHON/CS212_Project
$ python client.py
Public Key : (3, 101284087)
Session Key : TpuU142Bq82XHW
RSA Encrypted Session Key : [592704, 1404928, 1601613, 614125, 117649, 1406
08, 125000, 287496, 1442897, 175616, 125000, 681472, 373248, 658503]
[*] Session key established
Username: 
```

```
Pranay Machhi@Lenovo MINGW64 ~/OneDrive/Desktop/Coding/Programs 2
.0/Socket_Programming_PYTHON/CS212_Project
$ python server.py
Starting server... wait for it to initialize...
RSA Private Key: (67509307, 101284087)
RSA Public Key: (3, 101284087)
[*] Server listening on port 5000...
[+] Connected: ('127.0.0.1', 63830)
RSA Encrypted Session Key: [592704, 1404928, 1601613, 614125, 117
649, 140608, 125000, 287496, 1442897, 175616, 125000, 681472, 373
248, 658503]
RSA Decrypted Session Key: TpuU142Bq82XHW
[*] Session key established: TpuU142Bq82XHW
    This should be stored in Server memory securely so that
    noone can get this key
```

Symmetric Encryption using session key

All communication becomes encrypted as soon as the server and the client share a common key using symmetric encryption.

RSA helped to share this common session key across the network securely.

We couldn't use RSA to encrypt all the messages as long messages are costly (require too much time to be encrypted)

Message → Keystream Generation → XOR + Transformation → Ciphertext

KEYSTREAM GENERATION + XOR :

This is one of the algorithms used for symmetric key encryption

Benefits :

- fast communication
- lightweight
- suitable for continuous messaging

Multi-Threaded Server

Each client gets: (threading in socket programming)

- independent session
- independent encryption key
- independent authentication state

Role-Based Access Control (RBAC)

There are 2 roles :

1. User :

- normal commands
- utility functions (like PAL , LEN, ECHO)

2. Admin :

- view server statistics
- reset server stats

if (role != "admin"):

return "Unauthorized"

- Controlled access for Users

Supported Commands

Commands	Function
ECHO	return same message
REVERSE	reverse text
HASH	SHA256 hashing
TIME	current server time
RANDOM	random string
CALC	expression evaluation
WHOAMI	current role
STATS	server statistics
HELP	command list

FINAL OUTPUT

```
Pranay Machhi@Lenovo MINGW64 ~/OneDrive/Desktop/Coding/Programs 2.0/Socket_Programming_PYTHON/CS212_Project
$ python server.py
Starting server... wait for it to initialize...
[*] Server listening on port 5000...
[+] Connected: ('127.0.0.1', 64551)
[*] Session key established: noM0PRt7ODycVdnp3C
    This should be stored in Server memory securely
so that noone can get this key
[-] Disconnected: ('127.0.0.1', 64551)
[+] Connected: ('127.0.0.1', 56047)
[*] Session key established: 2qAQfh9xc3ZYndltN
    This should be stored in Server memory securely
so that noone can get this key
█

Pranay Machhi@Lenovo MINGW64 ~/OneDrive/Desktop/Coding/Programs 2.0/Socket_Programming_PYTHON/CS212_Project
$ python client.py
[*] Session key established
Username: admin
Password: admin123
AUTH SUCCESS

Available Commands:
    ECHO <msg>
    REVERSE <msg>
    TIME
    HASH <msg>
    UPPER <msg>
    LOWER <msg>
    LEN <msg>
    PAL <msg> (palindrome check)
    RANDOM <length>
    CALC <expression>
    WHOAMI
    STATS (admin only)
    CLEARSTATS (admin only)
    HELP
    EXIT
>> WHOAMI
Server: Role: admin
>> ECHO PRANAY_MACHHI
Server: PRANAY_MACHHI
>> HASH PRANAY_MACHHI
Server: 81a369361f6fa12517d41d2371d10c5adbcf0793ec203483e1634b0d252f7c5e
>> HASH PRANAY_MACHHL
Server: f325a903bd92fce365bd852bbd62a70cadfac577ac98b27199cfe2a4a74e1d70
```

Here i have logged in with dummy admin account and ran some commands

As shown in the server side we can see that there is a session key for each new connection with server-to-client

And that session key is generated at the client side and sent it to server

Only server could decrypt it with RSA private key

THANK YOU

MACHHI PRANAY B.

2403122